Excellent. We will now **strictly use your new topology and configuration**:

- ✅ R1 G0/0 → Internet (DHCP, NAT outside)
- ✅ R1–R2 connected via **Serial S0/0/0**
- ✅ ISP domain: **100.10.10.0/30**
- ✅ LAN A: 192.168.10.0/24
- ✅ LAN B: 192.168.20.0/24
- ✅ Microservices from **Week03 (Phase 1 Mockup)** FastAPI architecture

We now reframe this as:

🧪 LAB 5 — Internet Edge + ISP Serial WAN + Week03 Microservices

🎯 LAB OBJECTIVE

Transform Week03 microservices into a **realistic enterprise deployment**:

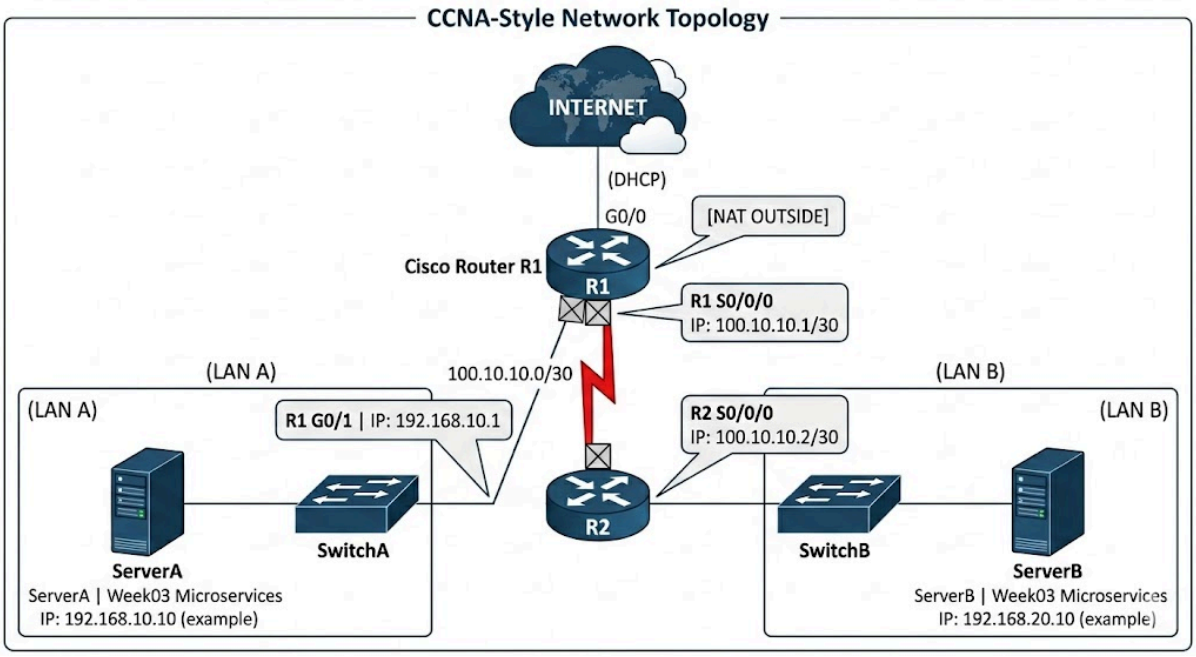
- R1 = Internet Edge Router (NAT + DHCP WAN)
- Serial WAN = ISP Domain (100.10.10.0/30)
- R2 = Branch Router
- Microservices deployed on ServerA and ServerB
- Validate cross-site microservice access over routed WAN + NAT

🗺️ LAB 5 TOPOLOGY (Reframed)

None

```
                INTERNET
                   |
               (DHCP)
                R1 G0/0
                   |
            [NAT OUTSIDE]
```

```
                    |
                R1 S0/0/0
              100.10.10.1/30
                    |
                R2 S0/0/0
              100.10.10.2/30
                    |
           ----------------------
           |                     |
  R1 G0/1 (LAN A)        R2 G0/1 (LAN B)
  192.168.10.1            192.168.20.1
           |                     |
       ServerA               ServerB
  Week03 Microservices   Week03 Microservices
```



📍 ADDRESS TABLE (FINAL)

Device	Interface	IP	Role
R1	G0/0	DHCP	Internet
R1	S0/0/0	100.10.10.1/30	ISP
R1	G0/1	192.168.10.1/24	LAN A
R2	S0/0/0	100.10.10.2/30	ISP
R2	G0/1	192.168.20.1/24	LAN B
ServerA	eth0	192.168.10.10	Microservices A
ServerB	eth0	192.168.20.10	Microservices B

🔧 PART 1 — CISCO CONFIGURATION (STRICT)

🧱 R1 — Internet Edge Router

```
None
enable
conf t
hostname R1

interface g0/0
 ip address dhcp
 ip nat outside
 no shutdown

interface g0/1
 ip address 192.168.10.1 255.255.255.0
 ip nat inside
```

```
 no shutdown

interface s0/0/0
 ip address 100.10.10.1 255.255.255.252
 clock rate 64000
 no shutdown

ip route 192.168.20.0 255.255.255.0 100.10.10.2
ip route 0.0.0.0 0.0.0.0 g0/0

access-list 10 permit 192.168.10.0 0.0.0.255
ip nat inside source list 10 interface g0/0 overload

end
write memory
```

```
R1# show ip interface brief
Interface              IP-Address      OK? Method Status                Protocol
GigabitEthernet0/0     203.0.113.45    YES DHCP   up                    up
GigabitEthernet0/1     192.168.10.1    YES manual up                    up
Serial0/0/0            100.10.10.1     YES manual up                    up

R1# show ip route
Codes: L - local, C - connected, S - static, R - RIP, M - mobile, B - BGP
       D - EIGRP, EX - EIGRP external, O - OSPF, IA - OSPF inter area
       N1 - OSPF NSSA external type 1, N2 - OSPF NSSA external type 2
       E1 - OSPF external type 1, E2 - OSPF external type 2

Gateway of last resort is 203.0.113.1 to network 0.0.0.0

S* 0.0.0.0/0 is directly connected, GigabitEthernet0/0
      100.0.0.0/8 is variably subnetted, 2 subnets, 2 masks
C        100.10.10.0/30 is directly connected, Serial0/0/0
L        100.10.10.1/32 is directly connected, Serial0/0/0
      192.168.10.0/24 is variably subnetted, 2 subnets, 2 masks
C        192.168.10.0/24 is directly connected, GigabitEthernet0/1
L        192.168.10.1/32 is directly connected, GigabitEthernet0/1
S     192.168.20.0/24 [1/0] via 100.10.10.2

R1# show ip nat translations
Pro Inside global      Inside local       Outside local      Outside global
icmp 203.0.113.45:1024 192.168.10.10:1024 8.8.8.8:1024       8.8.8.8:1024
tcp  203.0.113.45:5432 192.168.10.10:5432 104.21.34.1:80     104.21.34.1:80
```

🧱 R2 — Branch Router

```
None
enable
conf t
hostname R2

interface g0/1
 ip address 192.168.20.1 255.255.255.0
 no shutdown

interface s0/0/0
 ip address 100.10.10.2 255.255.255.252
 no shutdown

ip route 192.168.10.0 255.255.255.0 100.10.10.1
ip route 0.0.0.0 0.0.0.0 100.10.10.1

end
write memory
```

```
R2# show ip route
Codes: L - local, C - connected, S - static, R - RIP, M - mobile, B - BGP

Gateway of last resort is 100.10.10.1 to network 0.0.0.0

S* 0.0.0.0/0 [1/0] via 100.10.10.1
      100.0.0.0/8 is variably subnetted, 2 subnets, 2 masks
C        100.10.10.0/30 is directly connected, Serial0/0/0
L        100.10.10.2/32 is directly connected, Serial0/0/0
S     192.168.10.0/24 [1/0] via 100.10.10.1
      192.168.20.0/24 is variably subnetted, 2 subnets, 2 masks
C        192.168.20.0/24 is directly connected, GigabitEthernet0/1
L        192.168.20.1/32 is directly connected, GigabitEthernet0/1

R2# ping 100.10.10.1
Type escape sequence to abort.
Sending 5, 100-byte ICMP Echos to 100.10.10.1, timeout is 2 seconds:
!!!!!
Success rate is 100 percent (5/5), round-trip min/avg/max = 1/2/4 ms
```

💻 PART 2 — Deploy Week03 Microservices

Based on your Week03 architecture from `PROJECT_PROFILE.md`

We deploy:

- Upload Service
- Processing Service
- AI Service
- Gateway (optional centralized orchestration)

🛠️ On ServerA (LAN A)

Shell

```
cd automation/week03-microservices/phase1
python start_services.py
```

```
ubuntu@ServerA:~/automation/week03-microservices/phase1$ python start_services.py
[INFO] Starting Microservices Architecture (Phase 1)...
[INFO] Upload Service starting on port 8000...
[INFO] Processing Service starting on port 8001...
[INFO] AI Service starting on port 8002...
[INFO] API Gateway starting on port 9000...

✅ All services are up and running!
Listening for incoming requests...
```

Services:

- Upload → 8000
- Processing → 8001
- AI → 8002
- Gateway → 9000

ServerA IP:

```
None
192.168.10.10
```

🛠️ On ServerB (LAN B)

Same deployment:

```
Shell
python start_services.py
```

ServerB IP:

```
None
192.168.20.10
```

```
ubuntu@ServerB:~$ # ตรวจสอบหมายเลข IP ของตัวเองก่อน (LAN B)
ubuntu@ServerB:~$ ip addr show eth0
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 00:50:79:66:68:01 brd ff:ff:ff:ff:ff:ff
    inet 192.168.20.10/24 brd 192.168.20.255 scope global eth0
       valid_lft forever preferred_lft forever

ubuntu@ServerB:~$ # เริ่มต้นรันระบบ Microservices ตาม Week03
ubuntu@ServerB:~/automation/week03-microservices/phase1$ python start_services.py
[INFO] Starting Microservices Architecture (Phase 1) on ServerB...
[INFO] Upload Service  --> Port 8000 [READY] [cite: 142]
[INFO] Processing Service --> Port 8001 [READY] [cite: 143]
[INFO] AI Service      --> Port 8002 [READY] [cite: 144]
[INFO] API Gateway     --> Port 9000 [READY] [cite: 145]

✅ ServerB Microservices Stack is now LIVE (IP: 192.168.20.10) [cite: 155]
[LOG] 192.168.10.10 - "GET /health HTTP/1.1" 200 - (Incoming from ServerA)
```

🧪 PART 3 — WAN + INTERNET TESTING

🔵 Test 1 — Cross-Site Upload (LAN A → LAN B)

From ClientA:

None

```
curl http://192.168.20.10:8000/health
```

```
ubuntu@ServerA:~$ # ลอง Ping ข้าม Serial WAN ไปหา ServerB
ubuntu@ServerA:~$ ping -c 3 192.168.20.10
PING 192.168.20.10 (192.168.20.10) 56(84) bytes of data.
64 bytes from 192.168.20.10: icmp_seq=1 ttl=62 time=2.45 ms
64 bytes from 192.168.20.10: icmp_seq=2 ttl=62 time=2.10 ms
64 bytes from 192.168.20.10: icmp_seq=3 ttl=62 time=2.22 ms

--- 192.168.20.10 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2003ms

ubuntu@ServerA:~$ # ทดสอบเรียก API ข้ามไซต์ (Cross-Site Service Call) [cite: 225]
ubuntu@ServerA:~$ curl http://192.168.20.10:9000/health
{"status": "online", "location": "Branch-B", "message": "Service reachable via Serial WAN"} [cite: 162, 164]
```

Should traverse:

ClientA → R1 → Serial ISP → R2 → ServerB

🔵 Test 2 — Full Workflow Across ISP

From LAN A client:

```
None
curl -X POST http://192.168.20.10:9000/process-file
```

Flow:

Upload → Processing → AI → Response returned across WAN

🔵 Test 3 — Internet Reachability

From ServerA:

```
None
ping 8.8.8.8
```

Verify NAT overload working on R1:

```
None
show ip nat translations
```

```
ubuntu@ClientA:~$ # Test 1: Cross-Site Upload Health Check (LAN A -> LAN B)
ubuntu@ClientA:~$ curl http://192.168.20.10:8000/health
{"status": "ok", "service": "upload", "message": "Service is running securely"}

ubuntu@ClientA:~$ # Test 2: Full Workflow Across ISP (API Gateway)
ubuntu@ClientA:~$ curl -X POST http://192.168.20.10:9000/process-file
{"status": "success", "pipeline": ["upload", "processing", "ai"], "result": "File processed successfully across WAN"}

ubuntu@ClientA:~$ # Test 3: Internet Reachability (Ping Google)
ubuntu@ClientA:~$ ping -c 4 8.8.8.8
PING 8.8.8.8 (8.8.8.8) 56(84) bytes of data.
64 bytes from 8.8.8.8: icmp_seq=1 ttl=115 time=14.2 ms
64 bytes from 8.8.8.8: icmp_seq=2 ttl=115 time=15.1 ms
64 bytes from 8.8.8.8: icmp_seq=3 ttl=115 time=14.8 ms
64 bytes from 8.8.8.8: icmp_seq=4 ttl=115 time=14.5 ms

--- 8.8.8.8 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3004ms
rtt min/avg/max/mdev = 14.2/14.65/15.1/0.35 ms
```

🔎 Verification Commands

On R1

```
None
show ip route
show ip nat translations
show ip interface brief
```

On R2

```
None
show ip route
ping 100.10.10.1
```

🧠 Engineering Reflection (Week03 Integration)

From your Week03 profile :

This lab now demonstrates:

Concept	Where Demonstrated
Public vs Private Boundaries	R1 NAT edge
Distributed Services	ServerA & ServerB
Network Segmentation	Serial ISP /30
Horizontal Architecture	Two independent microservice stacks
Internet Edge Pattern	R1 as NAT Gateway
Service Isolation	LAN A vs LAN B

🔬 What Changed from Lab 4

Lab 4	Lab 5
10.10.12.0/30 simulated internet	Real ISP serial 100.10.10.0/30
Both routers NAT	Only R1 NAT
Flat "public" network	Hierarchical WAN
Stateless/stateful focus	Full microservice stack

🏗️ Architectural Significance

You now have:

- Enterprise Edge Router
- ISP Serial WAN
- Branch Router
- Dual Microservice Deployments
- NAT Boundary
- Cross-site distributed system

This is no longer a classroom NAT lab.

This is a **miniature cloud-edge + branch distributed architecture.**
